



南開大學
Nankai University

计算机学院
计算机体系结构实验报告

AXI 总线

姓名：林晖鹏

学号：2312966

专业：计算机科学与技术

2026 年 1 月 22 日

摘要

本实验围绕计算机体系结构中 AXI 总线的适配与验证展开，以六级流水线 CPU 为基础，完成了类 SRAM 总线改造、类 SRAM-AXI 转接桥实现，并验证了固定延迟与随机延迟下的总线交互效果。实验首先对原始 SRAM 接口扩展 size、addr_ok、data_ok 信号实现类 SRAM 总线，解决 CPU 与外设的时序解耦问题；随后设计转接桥完成类 SRAM 双端口到 AXI4 单拍主机接口的协议转换，包含读写请求仲裁、多通道信号映射、冲突检测与响应分流逻辑；最后通过控制 axi_wrap_ram.v 中的宏定义，分别验证了固定延迟（启用 RUN_PERF_TEST）和随机延迟（关闭宏定义）场景下的总线功能，94 个测试点均通过验证。实验结果表明，改造后的 CPU 可通过 AXI 总线实现稳定的访存交互，适配不同延迟特性的外设，同时深入理解了 AXI 总线通道分离、VALID-READY 握手机制的核心设计思想，以及总线适配过程中请求仲裁、冲突规避、数据缓冲的工程实现要点。

关键字：AXI 总线、流水线

目录

1 实验要求	3
2 实验原理	3
2.1 总线技术	3
2.2 类 SRAM 总线	3
2.2.1 核心架构	3
2.2.2 交互时序	3
2.2.3 对比 SRAM	4
2.3 AXI 总线	4
2.3.1 核心设计理念	4
2.3.2 关键信号与通道	4
2.3.3 基本传输流程	4
2.3.4 核心优势	5
3 实验流程	5
3.1 实验衔接	5
3.2 环境配置	5
3.3 类 SRAM 总线实现	6
3.3.1 mycpu_top.v 顶层模块	6
3.3.2 pre_if_stage.v 预取值模块	7
3.3.3 if_stage.v 取指模块	8
3.3.4 EXE_stage.v 执行模块	8
3.3.5 MEM_stage.v 访存模块	9
3.3.6 小结	10
3.4 类 SRAM-AXI2x1 转接桥实现	10
3.4.1 读请求处理流程 (SRAM→AXI AR→AXI R→SRAM)	10
3.4.2 写请求处理流程 (SRAM→AXI AW/W→AXI B→SRAM)	11
3.4.3 3. 冲突检测与请求记录	12

4 实验结果	13
4.1 随机延迟验证	13
5 实验心得	14
6 实验总结	14

1 实验要求

1. 根据自己情况，类 SRAM 总线和 AXI 总线完成一项支持即可，建议选择 AXI 总线（需要查阅 AXI 总线文档）。
2. 源码建议不再使用原始的五级流水线 CPU（如果使用也可，需要移植到现在的实验中去），源码请使用书中给的 lab10 或者 lab11 的源码。
3. 不要求做到多完美，随机延迟或者固定延迟的功能验证效果不强制要求，但需要把自己的探索思路、尝试的方法、效果和分析内容在实验报告里面写清楚。

2 实验原理

2.1 总线技术

在五级流水线 CPU 中，访存阶段是 CPU 与外部存储器/外设交互的核心环节。在原有的代码框架中，CPU 常直接连接 SRAM，通过「地址线 + 数据线 + 读写控制线」完成数据交互，但这种直连方式存在两个核心问题：

1. 扩展性差：CPU 只能连接单个 SRAM 设备，无法同时接入多个外设（如串口、定时器）；
2. 时序耦合紧：CPU 和 SRAM 必须严格同步工作，若外设速度慢（如低速 IO），会阻塞 CPU 流水线，降低执行效率。

总线技术的本质是 CPU 与外部设备之间的通用通信规则：CPU 只需按总线规则发送请求，外设按规则响应，无需关心对方的内部时序，既解决了多设备连接问题，又解耦了 CPU 与外设的时序依赖。

2.2 类 SRAM 总线

类 SRAM 总线是对传统 SRAM 直连方式的轻量级总线化改造，核心保留了 SRAM「地址 + 数据 + 读写控制」的基础交互逻辑，同时增加了简易的设备选择和时序适配机制，是从“直连”到“总线”的过渡形态。

2.2.1 核心架构

类 SRAM 总线的硬件架构包含三类信号线，兼容传统 SRAM 接口的同时扩展了设备适配能力：

- **地址线**：CPU 输出访存地址，总线将地址解码后分配至对应外设；
- **数据线**：双向传输，CPU 写操作时输出数据，读操作时接收外设返回数据；
- **控制线**：在传统「读使能、写使能」基础上，增加「设备选择」信号，每类外设对应独立的 CS 信号，总线通过地址解码生成 CS 信号，确保同一时间仅一个外设响应。

2.2.2 交互时序

类 SRAM 总线采用同步时序，与 CPU 时钟严格对齐，核心交互流程分为读、写两种：

- **写操作**：CPU 在时钟上升沿输出地址、写数据和 WE 信号，总线解码地址生成对应外设的 CS 信号，外设 CS 有效时锁存地址和数据，完成写操作；

- **读操作：**CPU 在时钟上升沿输出地址、RD 信号，总线解码生成 CS 信号，外设在 CS 有效时读取地址并返回数据，CPU 在下一个时钟上升沿锁存数据。

2.2.3 对比 SRAM

类 SRAM 总线相较于原始 SRAM 直连方式，核心变化是在保留「地址 + 数据 + 读写控制」基础交互逻辑的前提下，新增了基于地址解码的设备选择信号机制，实现了多外设的接入能力，但未改变 CPU 与外设的同步时序依赖，仅通过简化的等待周期插入适配低速外设，整体仍属于对原始 SRAM 接口的轻量化扩展而非彻底的时序解耦改造。

2.3 AXI 总线

AXI 属于高性能、高带宽的异步互连总线，专为复杂 SoC 设计，相比类 SRAM 总线，其核心优势是「高扩展性、时序解耦、突发传输支持」，是现代处理器外设交互的主流标准。

2.3.1 核心理念

AXI 总线基于「通道分离、事务独立、握手机制」设计，核心目标是解耦 CPU 与外设的时序依赖，提升数据传输效率：

- **通道分离：**将地址、数据、控制、响应等信号拆分为独立通道，各通道可并行传输，避免信号阻塞；
- **事务独立：**每个传输事务（如读一个 32 位数据）独立标识，外设可乱序响应，CPU 按标识重组数据；
- **握手机制：**采用「VALID-READY」异步握手机制，发送方输出 VALID 信号表示数据/地址有效，接收方输出 READY 信号表示准备就绪，双方均就绪时完成传输，无需严格时钟同步。

2.3.2 关键信号与通道

AXI 包含 5 个独立通道，各通道的核心功能如下：

- **读地址通道 (AR)：**负责由 CPU 向外设发送读操作的地址及突发传输长度等控制信息；
- **读数据通道 (R)：**负责由外设向 CPU 返回读操作对应的数据及传输状态（如成功/错误）；
- **写地址通道 (AW)：**负责由 CPU 向外设发送写操作的地址及突发传输长度等控制信息；
- **写数据通道 (W)：**负责由 CPU 向外设传输写操作对应的数据及字节有效标识；
- **写响应通道 (B)：**负责由外设向 CPU 反馈写操作的完成状态。

2.3.3 基本传输流程

以单次 32 位写操作（非突发）为例，AXI 总线的交互流程如下：

1. **写地址阶段：**CPU 在 AW 通道输出写地址 (AWADDR)，置位 AWVALID；外设准备就绪后置位 AWREADY，双方握手完成，地址传输生效；

2. **写数据阶段**: CPU 在 W 通道输出写数据 (WDATA) 和字节使能 (WSTRB), 置位 WVALID; 外设准备就绪后置位 WREADY, 握手完成, 数据传输生效;
3. **写响应阶段**: 外设完成数据写入后, 在 B 通道输出响应 (BRESP, 如 OKAY 表示成功), 置位 BVALID; CPU 准备就绪后置位 BREADY, 握手完成, 写事务结束。

读操作流程类似: CPU 先通过 AR 通道发送读地址, 外设接收后通过 R 通道返回读数据和响应, CPU 确认后完成读事务。

2.3.4 核心优势

相较于类 SRAM 总线, AXI 总线在扩展性、时序兼容性与传输效率上实现显著升级: 类 SRAM 总线仅能支持简单多外设接入, 且需通过手动规划地址段适配设备, 同时 CPU 与外设需严格同步时钟, 低速外设易阻塞流水线, 仅需新增地址解码与 CS 信号模块即可适配; 而 AXI 总线支持多主设备与多从设备灵活接入, 借助交叉开关实现地址路由与事务调度, 通过 **VALID-READY 握手机制**实现跨时钟域工作, 无需阻塞 CPU 流水线, 且 AXI4 版本支持最多 256 拍的突发传输, 大幅提升批量数据传输效率, 但相应适配复杂度更高, 需额外实现事务标识、握手机制、响应处理等逻辑, CPU 访存阶段需重构为符合 AXI 规范的事务发起逻辑, 外设也需实现完整的 AXI 从设备接口。

3 实验流程

3.1 实验衔接

由于使用了新代码, 但是之前的 lab9 代码并未实现, 我在 github 上面找了一个已完成的 lab9 代码, 拿到本实验来使用, 方便后续实验, 这里是来源链接: [here](#)。我是拿 init 的 commit 代码, 同时参考了后面 commit 对本 lab 的实现。

下面我来简要分析一下这个 base 流水线的实现。与之前我们实现的不同, 这是一个六级流水线 CPU (pre-IF/IF/ID/EXE/MEM/WB), 顶层 mycpu_top 连接各级前递/阻塞/异常/CP0 信号, 并对外暴露指令与数据的 SRAM 风格握手机制; 核心模块包括取指/预取阶段 (pre_IF_stage、IF_stage)、译码 (ID_stage)、执行与访存 (EXE_stage、MEM_stage, 内含数据总线读写与对齐处理)、回写 (WB_stage)、算术逻辑单元 (alu)、寄存器堆 (regfile)、异常与中断管理 (cp0), 以及全局参数定义与辅助函数 (mycpu.h、tools.v)。

在这个流水线下, 我们可以完成基础的实验配置。

3.2 环境配置

下面按实验指导书上面的步骤进行前期配置:

1. 将 func_lab9 拷贝到 CPU_CDE_AXI/soft/目录里面
2. 打开 cpu132_gettrace 工程, 重新定制 cpu132_gettrace 工程中的 inst_ram, 此时选择加载 func_lab9 的 coe。
3. 运行 cpu132_gettrace 工程的仿真 (进入仿真界面后, 直接点击 run all 等待仿真运行完成), 生成新的参考 Trace 文件 golden_trace.txt。

```

gettrace end!
——Succeed in generating trace file!
$finish called at time : 2131085 ns : File "C:/Users/lhppp/Desktop/lab4/cpu132_gettrace/testbench/tb_top.v" Line 221
run: Time (s): cpu = 00:01:17 ; elapsed = 00:01:07 . Memory (MB): peak = 1054.781 ; gain = 0.000

```

图 3.1: cpu132_gettrace 工程仿真运行完成

运行完之后，94 个测试点此时都通过。下面开始在 lab9 的 myCPU 上面实现 AXI 总线。

3.3 类 SRAM 总线实现

从指导书里面的分析，我们知道，将标准 SRAM 接口改造为类 SRAM 接口，只需要增加 3 个信号: size、addr_ok 和 data_ok。

类 SRAM 总线需完成请求发起、地址握手、数据缓冲、异常丢弃等全流程功能，这些功能分散在多个模块中协同实现，各模块核心职责如下：

- **mycpu_top.v (顶层)**: 对外暴露类 SRAM 接口；在异常/eret 时控制 ‘*_data_ok’ 信号的丢弃；连接各流水级握手信号；
- **pre_if_stage.v / if_stage.v**: 生成取指请求，处理 ‘addr_ok/data_ok’ 握手，实现数据缓冲与气泡处理；分支/异常时更新 PC；
- **EXE_stage.v**: 生成数据端访存请求 (req/wr/size/wstrb/addr/wdata)，记录地址握手状态，缓存返回数据，计算写掩码与数据对齐；
- **MEM_stage.v**: 完成加载数据的对齐/符号扩展，处理 LWL/LWR 指令，输出最终写回结果；阻塞流水线直到 ‘ms_data_ok’ 触发。

3.3.1 mycpu_top.v 顶层模块

类 SRAM 总线对外暴露指令、数据两个独立端口，各端口信号定义如下两个表所示，在原本的基础上添加三个信号：

表 1: 指令端口信号定义

信号名	功能描述
inst_sram_req	取指请求有效标识
inst_sram_wr	写使能（指令端恒为 0，只读）
inst_sram_size[1:0]	传输粒度（指令端恒为 2'h2，即 32bit）
inst_sram_wstrb[3:0]	写字节使能（指令端恒为 0）
inst_sram_addr[31:0]	字地址（低 2 位补 0 对齐）
inst_sram_wdata[31:0]	写数据（指令端恒为 0）
inst_sram_addr_ok	地址被外设接受的握手信号
inst_sram_rdata[31:0]	外设返回的读数据
inst_sram_data_ok	读数据返回的握手信号

表 2: 数据端口信号定义

信号名	功能描述
data_sram_req	访存请求有效标识
data_sram_wr	写使能（读为 0，写为 1）
data_sram_size[1:0]	传输粒度（字/半字/字节及 LWL/LWR 对齐后大小）
data_sram_wstrb[3:0]	字节写使能（依据 SW/SH/SB/SWL/SWR 生成）
data_sram_addr[31:0]	访存地址（LWL/SWL 强制对齐到字起始）
data_sram_wdata[31:0]	写数据（经对齐、掩码处理）
data_sram_addr_ok	地址被外设接受的握手信号
data_sram_rdata[31:0]	外设返回的读数据
data_sram_data_ok	读数据返回的握手信号

顶层丢弃机制（异常/ERET 刷新） CPU 执行异常（如中断、指令错误）或 ERET（异常返回）指令时，需要立即“刷新流水线”——丢弃当前流水线中所有未完成的指令，跳转到异常处理入口 / 原程序继续执行。但此时可能存在一种情况：CPU 已经向外设（如内存）发了访存请求（取指 / 读数据 / 写数据），外设还没返回数据（地址已接受，但 data_ok 还没拉回），这些“在路上”的请求如果后续返回数据，会被错误地送入已刷新的流水线，导致程序执行出错。

‘mycpu_top.v’ 中通过以下代码标记并屏蔽 flush 时未完成的请求，避免旧数据污染流水线。flush 时将“仍在执行的请求”标记为 discard，后续即使外设返回 ‘data_ok’，也会被屏蔽，无法进入流水线。

```

1 // inst side
2 if (ws_ex || ws_eret)
3     inst_sram_discard <= {pfs_inst_waiting, fs_inst_waiting};
4 assign inst_sram_data_ok_discard = inst_sram_data_ok && ~|inst_sram_discard;
5
6 // data side
7 if (ws_ex || ws_eret)
8     data_sram_discard <= {es_data_waiting, ms_data_waiting};
9 assign data_sram_data_ok_discard = data_sram_data_ok && ~|data_sram_discard;

```

3.3.2 pre_if_stage.v 预取值模块

取指请求与缓冲 ‘inst_sram_req’ 仅在无分支停顿、无异常且未持有挂起地址时发起；‘pfs_addr_ok_r’ 会记忆“地址已接受但 FS 未 allowin”的状态，支持跨拍握手；‘fs_inst_unable’ 过滤数据返回信号，避免重复取数；‘pfs_inst_buff’ 暂存 IF 阶段未就绪时的返回数据，‘pfs_inst_ok’ 确认数据就绪后向前推送。

发起取指请求：

```

1 assign inst_sram_req =
2     pfs_valid &&
3     !pfs_addr_ok_r &&
4     !(bd_done && br_stall) &&
5     !ws_eret && !ws_ex;
6 assign inst_sram_addr = {pfs_pc[31:2], 2'b0};

```


地址握手保持：

```

1 assign inst_sram_req =
2   pfs_valid &&
3   !pfs_addr_ok_r &&
4   !(bd_done && br_stall) &&
5   !ws_eret && !ws_ex;
6 assign inst_sram_addr = {pfs_pc[31:2], 2'b0};

```

数据返回缓冲：

```

1 if (pfs_addr_ok && pfs_inst_sram_data_ok && !fs_allowin) begin
2   pfs_inst_buff_valid <= 1'b1;
3   pfs_inst_buff      <= inst_sram_rdata;
4 end
5 assign pfs_inst_ok = pfs_inst_buff_valid || (pfs_addr_ok && pfs_inst_sram_data_ok);

```

3.3.3 if_stage.v 取指模块

取指完成条件：

```

1 assign fs_ready_go = fs_inst_ok;
2 assign fs_inst_ok = pfs_to_fs_inst_ok || fs_inst_buff_valid || (fs_valid &&
   inst_sram_data_ok);

```

数据返回缓冲：

```

1 assign fs_ready_go = fs_inst_ok;
2 assign fs_inst_ok = pfs_to_fs_inst_ok || fs_inst_buff_valid || (fs_valid &&
   inst_sram_data_ok);
3
4 assign fs\_inst\_waiting = fs\_valid && !fs\_inst\_ok; //用于上游暂停

```

3.3.4 EXE_stage.v 执行模块

数据请求、对齐与写掩码 该段代码中，‘no_store’ 信号用于在异常或 ERET 指令执行期间屏蔽新的访存请求发起，避免异常处理过程中产生无效访存；针对 LWL/SWL/LWR/SWR 指令，先将访存地址强制对齐到字起始位置，再通过 ‘data_sram_size’ 和 ‘data_sram_wstrb’ 限定有效操作字节范围；‘st_strb’ 和 ‘st_data’ 会根据指令类型与地址低位生成精准的写字节掩码和对齐后的写数据，确保仅对目标字节进行修改；‘es_addr_ok_r’ 寄存器则用于记录地址握手成功但 MEM 阶段未就绪（‘ms_allowin’ = 0）的状态，从而支持“地址先被外设接受、数据后续返回”的跨拍握手机制，保证访存流程的连续性。

```

1 assign data_sram_req =
2   es_valid &&
3   !es_addr_ok_r &&
4   (es_mem_we || es_mem_re) && !no_store;
5 assign data_sram_wr   = es_mem_we;

```

```

6 assign data_sram_size = ... // 基于指令类型与地址低位
7 assign data_sram_addr =
8   (es_inst_lwl || es_inst_sw1) ? {es_alu_result[31:2], 2'b0} : es_alu_result;
9 assign data_sram_wstrb = st_strb; // 由地址低位与 SB/SH/SW/SWL/SWR 生成
10 assign data_sram_wdata = st_data; // 已按掩码与对齐处理
11
12 always @(posedge clk) begin
13   if (data_sram_req && data_sram_addr_ok && !ms_allowin)
14     es_addr_ok_r <= 1'b1; // 地址先被接受
15   else if (ms_allowin)
16     es_addr_ok_r <= 1'b0;
17 end
18 assign es_addr_ok = (data_sram_req && data_sram_addr_ok) || es_addr_ok_r;

```

EXE 阶段数据缓冲 EXE 阶段通过 ‘es_data_buff’ 暂存下游未就绪时的返回数据;

```

1 assign es_data_waiting = es_valid && es_addr_ok && !es_data_ok;
2 assign es_data_ok = es_data_buff_valid || (es_addr_ok && es_data_sram_data_ok);
3 assign es_data = es_data_buff_valid ? es_data_buff : data_sram_rdata;

```

3.3.5 MEM_stage.v 访存模块

MEM 阶段数据对齐/扩展 MEM 阶段等待 ‘ms_data_ok’ 触发后向前推送数据, 完成 LB/LH/LWL 等指令的对齐与符号扩展; LWL/LWR 指令在 MEM 内依据地址低位重排数据, 保证写回结果正确; ‘ms_data_waiting’ 回传顶层, 用于 flush 时标记需丢弃的返回数据。

```

1 assign ms_ready_go = ms_wait_mem ? ms_data_ok : 1'b1;
2 assign ms_data_ok = ms_data_buff_valid || data_sram_data_ok;
3 // 对齐与扩展 (节选)
4 assign mem_half_ex[31:16] = {16{ ms_inst_lh & mem_half[15] }};
5 assign mem_byte_ex[31: 8] = {24{ ms_inst_lb & mem_byte[ 7] }};
6 // LWL/LWR 拼接
7 assign mem_left =
8   ( {32{mem_addr == 2'b00}} & {mem_word[ 7: 0], 24'b0} ) |
9   ( {32{mem_addr == 2'b01}} & {mem_word[15: 0], 16'b0} ) |
10  ( {32{mem_addr == 2'b10}} & {mem_word[23: 0],  8'b0} ) |
11  ( {32{mem_addr == 2'b11}} & mem_word[31: 0] );
12 assign mem_left_strb =
13   ( {4{mem_addr == 2'b00}} & 4'b1000 ) |
14   ( {4{mem_addr == 2'b01}} & 4'b1100 ) |
15   ( {4{mem_addr == 2'b10}} & 4'b1110 ) |
16   ( {4{mem_addr == 2'b11}} & 4'b1111 );
17
18 assign mem_right =
19   ( {32{mem_addr == 2'b00}} & mem_word[31: 0] ) |
20   ( {32{mem_addr == 2'b01}} & { 8'b0, mem_word[31: 8]} ) |
21   ( {32{mem_addr == 2'b10}} & {16'b0, mem_word[31:16]} ) |
22   ( {32{mem_addr == 2'b11}} & {24'b0, mem_word[31:24]} );

```

```

23 assign mem_right_strb =
24     ( {4{mem_addr == 2'b00}} & 4'b1111 ) |
25     ( {4{mem_addr == 2'b01}} & 4'b0111 ) |
26     ( {4{mem_addr == 2'b10}} & 4'b0011 ) |
27     ( {4{mem_addr == 2'b11}} & 4'b0001 );

```

3.3.6 小结

在上面改动中我们实现了类 SRAM 总线，通过 ‘req/addr_ok/data_ok’ 解耦时序，允许可变延迟与背压。取指/数据各自有缓冲和保持寄存，确保在下游不允或异常清空时不丢/错用数据。异常处理时通过丢弃标记过滤在途返回。

3.4 类 SRAM-AXI2x1 转接桥实现

这里 ‘transfer_bridge’ 转接桥是实现「类 SRAM 风格双端口」到「AXI4 单拍主机接口」的桥接模块，核心目标是将 CPU 侧简易的类 SRAM 握手机制 (req/addr_ok/data_ok) 转换为符合 AXI4 协议的多通道交互逻辑。在这个转接桥里，我们除了实现信号转换，还需要实现通路仲裁。

3.4.1 读请求处理流程 (SRAM→AXI AR→AXI R→SRAM)

读请求处理是模块核心逻辑，涵盖 “请求仲裁 → 地址生成 → 响应分流 → 数据返回” 四个阶段：

阶段 1：读请求仲裁 (数据优先) 模块接收指令端口 (inst_sram_req) 和数据端口 (data_sram_req) 的读请求，按 “数据优先” 原则仲裁：

```

1 assign read_req_sel_data    = data_read_valid;
2 assign read_req_sel_inst    = !data_read_valid && inst_read_valid;
3 assign read_req_valid      = inst_read_valid || data_read_valid;
4 assign read_req_id         = read_req_sel_data ? DATA_ID : INST_ID;

```

阶段 2：AXI 读地址 (AR) 通道生成 通过 ‘axi_ar_busy’ 状态机锁存仲裁后的读请求，生成 AXI AR 通道信号：

```

1 always @ (posedge aclk) begin
2     if (!aresetn) begin
3         axi_ar_busy <= 1'b0;
4         axi_ar_id   <= 4'h0;
5         axi_ar_addr <= 32'h0;
6         axi_ar_size <= 3'h0;
7     end else if (!axi_ar_busy && read_req_valid) begin
8         axi_ar_busy <= 1'b1; // 锁存请求，置位 arvalid
9         axi_ar_id   <= read_req_id;
10        axi_ar_addr <= read_req_addr;
11        axi_ar_size <= read_req_size;
12    end else if (axi_ar_busy && arvalid && arready) begin
13        axi_ar_busy <= 1'b0; // 地址握手完成，释放状态
14    end

```

```

15 end
16 assign arvalid = axi_ar_busy;

```

阶段 3: AXI 读响应 (R) 通道分流 从机返回读数据 ('rvalid=1') 时, 根据 'rid' (事务 ID) 将数据分流至对应 FIFO:

```

1 assign axi_r_data_ok = rvalid && rready && rid == DATA_ID;
2 assign axi_r_inst_ok = rvalid && rready && rid == INST_ID;
3 assign rready = !read_inst_resp_full && !read_data_resp_full;
4
5 // 指令读响应FIFO写入
6 assign read_inst_resp_wen = axi_r_inst_ok;
7 assign read_inst_resp_input = axi_r_data;
8
9 // 数据读响应FIFO写入
10 assign read_data_resp_wen = axi_r_data_ok;
11 assign read_data_resp_input = axi_r_data;

```

阶段 4: SRAM 侧读数据返回 通过 FIFO 状态生成 SRAM 侧 'data_ok' 信号, 完成读事务:

```

1 // 指令端口: FIFO非空即返回data_ok
2 assign inst_sram_data_ok = !read_inst_resp_empty;
3 assign inst_sram_rdata = read_inst_resp_output;
4
5 // 数据端口: FIFO非空且请求记录匹配时返回data_ok
6 assign data_sram_data_ok = (data_read_ready && !read_data_resp_empty) || ...;
7 assign data_sram_rdata = read_data_resp_output;

```

3.4.2 写请求处理流程 (SRAM→AXI AW/W→AXI B→SRAM)

写请求仅支持数据端口, 采用 “AW/W 通道独立握手、B 通道计数确认” 的流程:

阶段 1: AXI 写地址 (AW) /写数据 (W) 通道生成 模块同时锁存写地址和写数据, 生成 AW/W 通道信号 (单拍传输):

```

1 always @ (posedge aclk) begin
2     // AW通道状态机
3     if (!aresetn) begin
4         axi_aw_busy <= 1'b0;
5         axi_aw_addr <= 32'h0;
6     end else if (!axi_aw_busy && !axi_w_busy && write_req_valid) begin
7         axi_aw_busy <= 1'b1; // 锁存写地址, 置位awvalid
8     end else if (axi_aw_busy && awvalid && awready) begin
9         axi_aw_busy <= 1'b0; // AW握手完成
10    end
11
12    // W通道状态机

```

```

13     if (!aresetn) begin
14         axi_w_busy <= 1'b0;
15         axi_w_data <= 32'h0;
16     end else if (!axi_aw_busy && !axi_w_busy && write_req_valid) begin
17         axi_w_busy <= 1'b1;    // 锁存写数据, 置位wvalid
18     end else if (axi_w_busy && wvalid && wready) begin
19         axi_w_busy <= 1'b0;    // W握手完成
20     end
21 end
22 assign awvalid  = axi_aw_busy;
23 assign wvalid   = axi_w_busy;

```

- 触发条件：仅当 ‘write_req_valid’（数据端口写请求有效）且 AW/W 通道空闲时，锁存请求；
- 独立握手：AW 通道（地址）和 W 通道（数据）分别与从机握手，无需严格同步；
- 生成 ‘addr_ok’：AW/W 通道锁存请求后，向 CPU 返回 ‘data_sram_addr_ok’。

阶段 2：AXI 写响应 (B) 通道处理 从机返回写响应（‘bvalid=1’）时，通过计数 FIFO 记录响应，生成写完成信号：

```

1 assign axi_b_ok = bvalid && bready;
2 assign bready = !write_data_resp_full;
3 assign write_data_resp_wen = axi_b_ok;    // 响应入队
4
5 // 写完成确认：计数FIFO非空且请求记录匹配
6 assign data_write_ready = !data_req_record_empty && data_req_record_output[32];
7 assign data_sram_data_ok = (data_write_ready && !write_data_resp_empty) || ...;

```

- ‘bready’控制：仅当写响应计数 FIFO 非满时，置位 ‘bready’接收响应；
- 响应计数：通过深度为 6 的 FIFO 记录写响应数量，确保每个写请求都有对应的响应；
- 生成 ‘data_ok’：当计数 FIFO 非空且请求记录匹配时，置位 ‘data_sram_data_ok’，通知 CPU 写完成。

3.4.3 3. 冲突检测与请求记录

为避免数据端口同地址读写冲突，模块通过 ‘data_req_record’ FIFO 记录在途请求：

```

1 // 请求入队：地址握手完成时，记录{写使能, 地址}
2 assign data_req_record_wen = data_sram_req && data_sram_addr_ok;
3 assign data_req_record_input = {data_sram_wr, data_sram_addr};
4
5 // 冲突检测：新请求地址与在途请求地址匹配时，置位data_related
6 assign data_related        = data_req_record_related_1;
7 assign data_read_valid     = data_sram_req && !data_sram_wr && !data_related;
8 assign data_write_valid    = data_sram_req && data_sram_wr && !data_related;

```

- 请求记录：每一个数据端口的有效请求（‘addr_ok’）都会被记录到 FIFO 中，包含“写使能 + 地址”；
- 冲突检测：新请求地址与 FIFO 中在途请求地址匹配时，‘data_related=1’，屏蔽该请求；
- 请求释放：当 ‘data_sram_data_ok’置位（事务完成）时，FIFO 出队，释放对应请求记录。

4 实验结果

至此，按照指导书上面的思路，我们实现了由类 SRAM 接口的 CPU 以及类 SRAM-AXI 转接桥组成的一个 AXI 接口的 CPU。下图是在固定随机延迟下的测试结果：

```

——[5482885 ns] Number 8'd93 Functional Test Point PASS!!!
      [5492000 ns] Test is running, debug_wb_pc = 0xbfc006b0
      [5502000 ns] Test is running, debug_wb_pc = 0xbfc004f8
      [5512000 ns] Test is running, debug_wb_pc = 0xbfc9eae4
      [5522000 ns] Test is running, debug_wb_pc = 0xbfc0039c
——[5523685 ns] Number 8'd94 Functional Test Point PASS!!!

Test end!
——PASS!!!
$finish called at time : 5525005 ns : File "C:/Users/lhppp/Desktop/lab4-2/mycpu_axi_verify/testbench/mycpu_tb.v" Line 267

```

图 4.2: 固定延迟的测试通过图

4.1 随机延迟验证

观察 ‘axi_wrap_ram.v’ 文件中的代码可以知道，当宏定义了 ‘RUN_PERF_TEST’ 时候，使用固定间隔的 backpressure (r2r、b2b 定时器)，当没有定义的时候，生成随机间隔延迟。这里我们删去开头了宏定义，然后进行随机延迟验证，并尝试修改 config.v 中的随机种子。

```

1 // mask: 为性能/鲁棒性测试引入 backpressure 掩码
2 #ifndef RUN_PERF_TEST
3     assign ar_and = 1'b1;
4     assign r_and = pf_r2r_nomask;
5     assign aw_and = 1'b1;
6     assign w_and = 1'b1;
7     assign b_and = pf_b2b_nomask;
8 else
9     assign ar_and = ram_random_mask[4] | ar_nomask;
10    assign r_and = ram_random_mask[3] ;
11    assign aw_and = ram_random_mask[2] | aw_nomask;
12    assign w_and = ram_random_mask[1] | w_nomask;
13    assign b_and = ram_random_mask[0] ;
14 endif

```

开启随机延迟之后，我们关注 inst_sram_ram 和 arvalid 的信号，发现这两个信号的上升沿不是一起的，并且间隔长度不固定，可以知道我们开启了随机延迟。

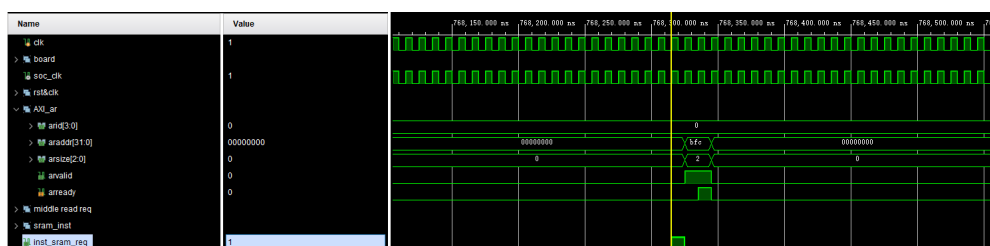


图 4.3: 随机延迟演示

然后下图是测试通过的截图：

```

-----[5523685 ns] Number 8' d94 Functional Test Point PASS!!!
-----
Test end!
-----PASS!!!
$finish called at time : 5525005 ns : File "C:/Users/lhppp/Desktop/lab4-2/mycpu_axi_verify/testbench/mycpu_tb.v" Line 267

```

图 4.4: 随机延迟测试通过

5 实验心得

- 本次实验改动变化很大，原理也不少，没有参考容易改的很乱也不知道怎么 debug，有网上的参考感觉好做一些，一开始自己写花费了很久时间，由于上下文还有点长，ai 也不好帮忙。
- 还有就是每次改动跑一次需要花费很久时间，debug 成本相当之大！
- 然后指导书上面有一些对一些值的固定，再一开始对这些信号不是很理解的情况下，网上的代码、ai 的代码再这些细节上面往往和指导书的不一样，一开始卡再这些错误地方比较久。
- 经过本次实验，对总线的概念以及类 SRAM 和 AXI 总线有了更深刻的理解。

6 实验总结

本次实验以六级流水线 CPU 为基础完成了 AXI 总线的适配与验证，核心完成类 SRAM 总线改造和类 SRAM-AXI 转接桥设计两大关键工作：通过扩展 ‘addr_ok’/‘data_ok’ 等信号实现类 SRAM 总线，解耦 CPU 与外设的同步时序依赖；基于 ‘transfer_bridge’ 模块完成类 SRAM 双端口到 AXI4 协议的转换，实现读写请求仲裁、多通道信号映射与冲突检测，并通过控制 ‘axi_wrap_ram.v’ 宏定义验证了固定/随机延迟下的总线功能，94 个测试点全通过。

实验不仅验证了 AXI 总线适配的可行性，也深化了对总线技术的认知：类 SRAM 总线是 SRAM 直连的轻量化扩展，而 AXI 总线通过通道分离、异步握手机制实现高扩展性，但其协议复杂度更高；同时掌握了异步握手的状态保持、数据缓冲及异常场景下请求丢弃的工程实现要点，为复杂 SoC 总线设计奠定了基础，也通过分段验证的方式有效降低了长周期仿真的调试成本。